

Section 02: Variables and Data Types Concept Map

The full path through this section - we open here before each concept, then dive in.



Variables: Keywords Covered

Section 02: Variables and Data Types - the keywords in bright green are the new ones we cover here.

Values True False None	Classes class	Context Managers with
Logic & Membership and or not in is	Error Handling try except finally raise assert	Cleanup & Flow pass del
Conditionals if elif else	Modules & Imports import from as	Generators & Async yield async await
Loops for while break continue	Scope global nonlocal	Soft Keywords match case _ type
Functions def return lambda		

keyword

 New in this section

keyword

 Covered earlier

keyword

 Not covered yet

Variables: First Program and Comments

Write your first line of Python with **print()**, and use **comments** - notes that Python ignores - to explain your code.

#	CONCEPT	SAMPLE CODE
1	Your first program <code>print()</code> print() writes text out to the screen.	<code>print("I am learning Python today")</code>
2	Comment line <code>#</code> A line that starts with # is ignored by Python.	<code># explain your code in plain words</code>
3	Inline comment Add a quick note at the end of a code line.	<code>print("Hi") # a note for people</code>
4	Comment out a line Put # in front to switch a line off without deleting it.	<code># print("this line will not run")</code>
5	Multi-line comment <code>"""</code> Triple quotes hold a note that spans several lines.	<code>"""a longer note over many lines"""</code>

Variables: The `print()` Function

`print()` shows values on the screen - your first tool for seeing results and finding bugs. In this step you learn everything one `print()` call can do.

#	CONCEPT	SAMPLE CODE
1	Print several values Commas join every value with one space.	<code>print("Python", "is", "Fun")</code>
2	Custom separator <code>sep=</code> Change what goes between the values.	<code>print("2026", "01", "01", sep="-")</code>
3	Custom line ending <code>end=</code> Default ending is a new line; change it here.	<code>print("same", end=" ")</code>
4	Blank line <code>print()</code> No arguments prints an empty line.	<code>print()</code>
5	New line <code>\n</code> Breaks one string onto the next line.	<code>print("Line One\nLine Two")</code>
6	Tab <code>\t</code> Inserts a tab to line up columns.	<code>print("Name:\tKalyan")</code>
7	Backslash and quotes <code>\\</code> <code>\"</code> Escape a backslash or a quote to print it.	<code>print("C:\\Users\\Kalyan")</code>

Variables: **Creating** and Naming (1 of 2)

A variable is a name attached to a value with =, so you can store data once and reuse it throughout your program.

#	CONCEPT	SAMPLE CODE
1	Create a variable <code>=</code> Attach a value to a name using =.	<code>name = "Kalyan"</code>
2	Show a variable <code>print()</code> <code>print()</code> displays the value a name holds.	<code>print(name)</code>
3	Text with a variable Show a label and its value in one line.	<code>print("age is", age)</code>
4	Reassign a variable <code>=</code> Assign again with = to update the value.	<code>age = 31</code>

Variables: **Creating** and Naming (2 of 2)

A variable is a name attached to a value with =, so you can store data once and reuse it throughout your program.

#	CONCEPT	SAMPLE CODE
5	Legal name rules Start with a letter or _, then letters, digits, or _.	<code>first_name = "Kalyan"</code>
6	snake_case style <code>snake_case</code> Lowercase words joined by underscores.	<code>favorite_color = "blue"</code>
7	Constants <code>UPPER_CASE</code> <code>UPPER_CASE</code> marks a value that never changes.	<code>MAX_SCORE = 100</code>
8	Case sensitive names age and Age are two different variables.	<code>print("age is", age, "and Age is", Age)</code>

Variables: Data Types and `type()`

Every value has a data type - str, int, float, or bool - and `type()` tells you which one a value is.

#	CONCEPT	SAMPLE CODE
1	Four core data types Text is str, whole number int, decimal float, True/False bool.	<pre>title = "Python Basics"</pre>
2	Check a value's type <code>type()</code> <code>type(value)</code> returns the data type of a value.	<pre>print(type(lessons))</pre>
3	Short type name <code>.__name__</code> Get the plain name like str, not the full class text.	<pre>print(type(rating).__name__)</pre>
4	Boolean values <code>True</code> <code>False</code> Only True or False, each with a capital first letter.	<pre>is_member = True</pre>
5	Round a decimal <code>round()</code> <code>round(value, digits)</code> shortens a long decimal number.	<pre>print(round(pi, 2))</pre>

Variables: Type Conversion

Type conversion changes a value from one type to another - text into numbers with `int()` and `float()`, numbers into text with `str()` - so your data is the right type for the job.

#	CONCEPT	SAMPLE CODE
1	Text to integer <code>int()</code> Turn typed text like "25" into a whole number.	<pre>age = int(age_text)</pre>
2	Text to decimal <code>float()</code> Turn text like "19.99" into a decimal number.	<pre>price = float(price_text)</pre>
3	Number to text <code>str()</code> Join a number into a text sentence without an error.	<pre>print("I have " + str(count) + " items")</pre>
4	Input is always text <code>input()</code> <code>input()</code> hands back a str, so <code>a + b</code> joins them.	<pre>a = input("Enter value for a: ")</pre>
5	Convert then compute <code>int(input())</code> Wrap <code>input()</code> in <code>int()</code> first to do real math.	<pre>a = int(input("Enter value for a: "))</pre>

Variables: Python Keywords

Keywords are words Python reserves for its own grammar, so you cannot use them as variable names.

#	CONCEPT	SAMPLE CODE
1	The reserved word list <code>keyword.kwlist</code> The keyword module holds Python's full list of keywords.	<pre>print(keyword.kwlist)</pre>
2	A keyword as a name fails Naming a variable after a keyword is a <code>SyntaxError</code> .	<pre>class = "Math"</pre>
3	Test a word <code>iskeyword()</code> Check whether a word is a keyword before you use it.	<pre>keyword.iskeyword("for")</pre>